

01 / AUTONOMOUS ROBOTICS

Ultrasonic Obstacle Avoider Robot

| Non-Contact Distance Sensing & Spatial Avoidance with ESP32.

ACTUATOR 26

FIELD NOTES / 2026

02 / SYSTEM ARCHITECTURE

4 Core Modules of an Obstacle Avoider

Real-time distance measurement drives autonomous path clearing:

BRAIN

ESP32 Microcontroller

Triggers 40kHz sonic pulses on GPIO 32, measures microsecond echo pulse widths on GPIO 35, and computes distance in centimeters.

PERCEPTION

HC-SR04 Ultrasonic Sensor

Single front-facing ultrasonic module provides non-contact distance sensing from 2cm to 400cm with 3mm precision.

ACTUATION

TB6612FNG Driver

Dual H-Bridge MOSFET driver controlling left and right DC motors to move forward or pivot away from obstacles.

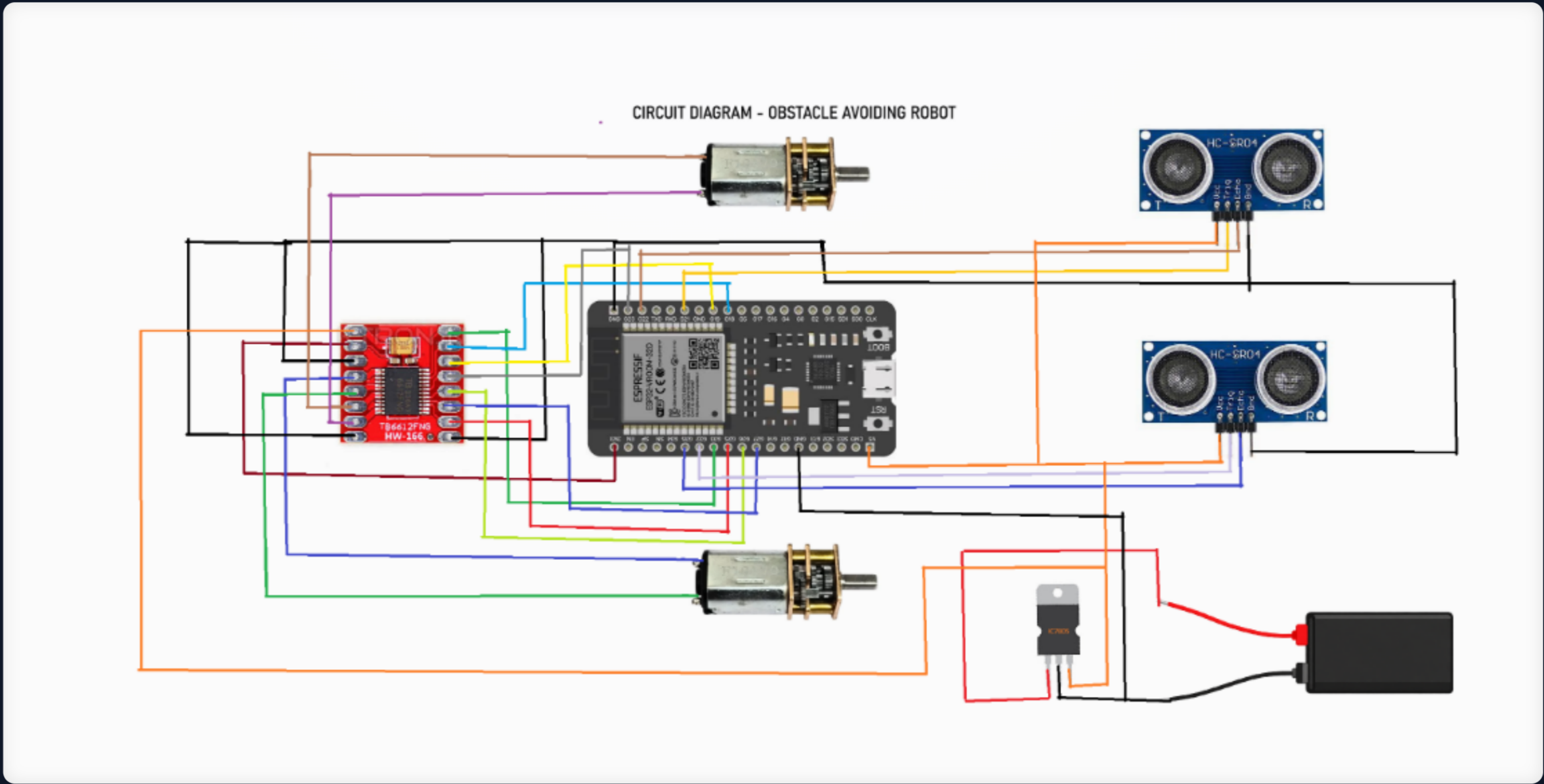
POWER

Isolated LiPo Rail

Delivers clean logic power to ESP32 and high-current motor supply (VM) to avoid brownouts during directional changes.

03 / SCHEMATIC DIAGRAM

Circuit Diagram & Hardware Interconnects



Ultrasonic Sensor
Trig: 32 | Echo: 35

TB6612FNG Driver
STBY: 23 | Left: 33/18/19 | Right: 25/26/27

Threshold Distance
20.0 cm

Motor Speed
100 PWM

04 / SENSOR PRINCIPLE

HC-SR04 Ranging Sonic Echo Principle

Computing target distance from sound propagation velocity (343 m/s):

-  **Trigger Pulse**
Send 10µs HIGH pulse on RC_TRIG to fire 8-cycle 40kHz sonic burst.
-  **Echo Measurement**
pulseIn(RC_ECHO, HIGH) measures microsecond travel duration.

$$\text{Distance (cm)} = \frac{\text{Duration } (\mu\text{s}) \times 0.0343}{2}$$

// Distance Measurement Helper Function

```
float measureDistance(int trigPin, int echoPin)
{
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    long duration = pulseIn(echoPin, HIGH);
    float distance = (duration * 0.0343) / 2; // Convert to cm
    return distance;
}
```



05 / PIN MAPPING

ESP32 Pinout Assignment

Obstacle Avider Config

Exact GPIO assignments configured in `obstacle_avoider/src/main.cpp`:

ULTRASONIC SENSOR PINS

RC_TRIG	GPIO 32 (Output)
RC_ECHO	GPIO 35 (Input)
THRESHOLD_DISTANCE	20.0 cm
SPEED	100 (PWM Duty Cycle)

MOTOR DRIVER PINS

RC_MOTOR_STBY	GPIO 23 (Standby Enable)
RC_LEFT_PWM / IN1 / IN2	GPIO 33 / 18 / 19
RC_RIGHT_PWM / IN1 / IN2	GPIO 25 / 26 / 27



06 / CONTROL ALGORITHM

Obstacle Avoidance

State Decision Matrix

Measured distance comparison against THRESHOLD_DISTANCE (20.0 cm):

MEASURED DISTANCE	CONDITION	LEFT MOTOR STATE	RIGHT MOTOR STATE	ROBOT BEHAVIOR
> 20.0 cm	Clear Path	FORWARD (IN1: HIGH, IN2: LOW, PWM 100)	FORWARD (IN1: HIGH, IN2: LOW, PWM 100)	Move Straight Forward
≤ 20.0 cm	Obstacle Detected	FORWARD (IN1: HIGH, IN2: LOW, PWM 100)	REVERSE (IN1: LOW, IN2: HIGH, PWM 100)	Pivot Right Turn (Avoid Obstacle)

07 / CODE BREAKDOWN

Initialization & Ultrasonic Setup Code

Configuring Trig/Echo pins & motor driver enable in setup():

```
// setup() from obstacle_avoider/src/main.cpp  
  
void setup()  
{  
    Serial.begin(115200);  
    pinMode(RC_MOTOR_STBY, OUTPUT);  
    pinMode(RC_LEFT_PWM, OUTPUT);  
    pinMode(RC_LEFT_IN1, OUTPUT);  
    pinMode(RC_LEFT_IN2, OUTPUT);  
    pinMode(RC_RIGHT_PWM, OUTPUT);  
    pinMode(RC_RIGHT_IN1, OUTPUT);  
    pinMode(RC_RIGHT_IN2, OUTPUT);  
  
    pinMode(RC_TRIG, OUTPUT);  
    pinMode(RC_ECHO, INPUT);  
  
    digitalWrite(RC_MOTOR_STBY, HIGH);  
}
```

08 / CODE BREAKDOWN

Distance Measurement & Serial Telemetry

Sampling ultrasonic sensor distance in loop():

```
// Distance Sampling in loop()  
  
void loop()  
{  
    float distance = measureDistance(RC_TRIG, RC_ECHO);  
    Serial.print("Distance: ");  
    Serial.println(distance);  
  
    // Obstacle Avoidance Decision Tree follows...
```

09 / CODE BREAKDOWN

Obstacle Avoidance Decision Logic Code

Executing directional movement based on threshold distance:

```
if(distance > THRESHOLD_DISTANCE)
{
    // Distance > 20cm -> Move Forward
    digitalWrite(RC_LEFT_IN1, HIGH);
    digitalWrite(RC_LEFT_IN2, LOW);
    analogWrite(RC_LEFT_PWM, SPEED);

    digitalWrite(RC_RIGHT_IN1, HIGH);
    digitalWrite(RC_RIGHT_IN2, LOW);
    analogWrite(RC_RIGHT_PWM, SPEED);
}
else
{
    // Distance <= 20cm -> Pivot Turn (Left Forward, Right Reverse)
    digitalWrite(RC_LEFT_IN1, HIGH);
    digitalWrite(RC_LEFT_IN2, LOW);
    analogWrite(RC_LEFT_PWM, SPEED);

    digitalWrite(RC_RIGHT_IN1, LOW);
    digitalWrite(RC_RIGHT_IN2, HIGH);
    analogWrite(RC_RIGHT_PWM, SPEED);
}
```

10 / HARDWARE TUNING

Threshold Tuning & Sensor Calibration

DISTANCE THRESHOLD**20.0 cm Threshold**

Provides ample detection margin at 100 PWM speed, preventing physical collisions with obstacles.

PWM SPEED**100 Duty Cycle**

Provides stable motor traction and controlled pivot maneuverability during obstacle avoidance.

LOOP INTERVAL**1000ms Delay**

`delay(1000)` at the end of `loop()` creates 1-second sampling cycles between navigation steps.

GPIO 35 INPUT ONLY**ESP32 Pin Constraint**

GPIO 35 is an input-only pin without internal pull-ups, making it ideal for the HC-SR04 ECHO pulse.

11 / TROUBLESHOOTING

4 Ultrasonic Field Diagnostics

ISSUE 1**Soft Surface Absorption**

Fabric or foam surfaces absorb 40kHz acoustic waves. Test obstacle detection against hard, flat surfaces.

ISSUE 2**5V to 3.3V Echo Level Shift**

HC-SR04 Echo pin outputs 5V. Use a resistor voltage divider (1k/2k) before feeding to ESP32 3.3V GPIO 35.

ISSUE 3**Motor Supply Noise**

DC motor electrical noise on VM rail can trigger false sonar readings. Add a 100 μ F decoupling capacitor across VM/GND.

ISSUE 4**Timeout Protection**

If no echo reflection returns, pulseIn times out to avoid blocking execution indefinitely.



12 / READY TO AVOID

Sense. Decide. Avoid.

Your ESP32 Ultrasonic Obstacle Avider is configured for autonomous distance sensing and avoidance.

HC-SR04 · ESP32 · ACTUATOR 26

LET'S
AVOID IT